RAG PROJECT REPORT

Name	ID
ساندرا هاني سمير جبرة	2023170256
مريم دسوقي أحمد محمود	2023170584
مريم محمد محسن ابراهيم	2023170594
تسنيم مأمون شله	2022170723
مريم عبدالسلام الشحات عبدالسلام	2023170576
مروه احمد محمد سيد	2023170575

Research Question

Can a Retrieval-Augmented Generation system accurately match job candidates to job requirements by querying raw, unstructured multilingual documents (PDFs, DOCX, HTML) without any manual data cleaning or pre-labeling?

Querying Problem

The HR and recruitment domain produces large volumes of unstructured documents — CVs in various formats, job descriptions scraped from web pages, Word documents from hiring managers — in multiple languages including Arabic and English. A recruiter trying to answer questions like "which candidates have Python experience" or "what certifications does this role require" must manually read every document. This does not scale.

Our system addresses this by building an end-to-end RAG pipeline that ingests raw messy documents as-is, chunks and embeds them into a searchable vector database, and answers natural language queries in both Arabic and English by retrieving the most relevant document chunks and injecting them into an LLM context window.

Data Ingestion & Multilingual Processing Pipeline

1. Multi-Format Document Loading

- **Engineered** a robust loading system using pdfplumber for raw PDFs, python-docx for Word documents, and BeautifulSoup4 for HTML files.
- **Avoided "Toy Data":** The pipeline was designed to handle messy, unstructured documents directly, adhering to the "Dirty Hands" rule of the project.

2. Custom Data Cleaning & Standardization

- **Text Cleaning:** Implemented regex-based cleaning to remove noise, redundant whitespaces, and non-printable characters while preserving semantic punctuation.
- **Standardized Output:** Developed a uniform dictionary output format (JSON-ready) that includes the cleaned content and essential metadata (source name, file type, and character count).

3. Advanced Metadata Handling

- Designed the system to automatically extract and attach metadata to every document, ensuring that downstream components (like the Vector Database) can track the source of each information chunk.

4.Multilingual & Arabic Support

- **RTL Text Extraction (Logical Ordering):** Solved the common "Visual Order" issue in Arabic PDFs. By using arabic-reshaper and python-bidi, the system reorders Arabic characters into a **Logical Order** that LLMs and Embedding models can actually understand.
- **Script Normalization:** Wrote custom logic to normalize Arabic characters (e.g., unifying different shapes of Alif آ، إ، أ and Teh Marbuta ة). This significantly increases retrieval accuracy for Arabic queries.
- **Diacritics Removal:** Added an automated step to strip Arabic Tashkeel (diacritics) to ensure that the search process is not affected by varied vowel markings.

5. Retrieval & Embedding Specifications

To optimize the matching process between Job Descriptions and Resumes, the retrieval module was engineered with a focus on semantic orientation:

Vector Infrastructure: The system utilizes Python 3.11 with FAISS as the high-performance Vector Store. Embeddings are generated using the all-MiniLM-L6-v2 transformer-based model, chosen for its efficiency in mapping multilingual semantic relationships into a 384-dimensional vector space.

Similarity Metric (Cosine Similarity): We explicitly implemented Cosine Similarity to measure the relationship between queries and document chunks. Unlike Euclidean distance, which measures physical proximity, Cosine Similarity focuses on the angle between vectors. This ensures that the length of a CV does not penalize a candidate; the system prioritizes the "direction" of skills and conceptual overlap, ensuring a score close to 1 when the semantic intent aligns.

Strategic Chunking Logic:

Chunk Size (200 Words): This size acts as a "Semantic Window," balancing information density and signal. It is large enough to capture a full professional

experience without "Vector Dilution" (noise), yet small enough to fit within the model's token limits without truncation.

Overlap (20 Words): A 10% overlap is maintained to ensure "Boundary Integrity." This sliding window prevents the loss of information where key phrases (e.g., "Senior Software Engineer") might otherwise be split between two chunks, maintaining semantic continuity across the vector space.

Phase 4: Evaluation & Error Analysis

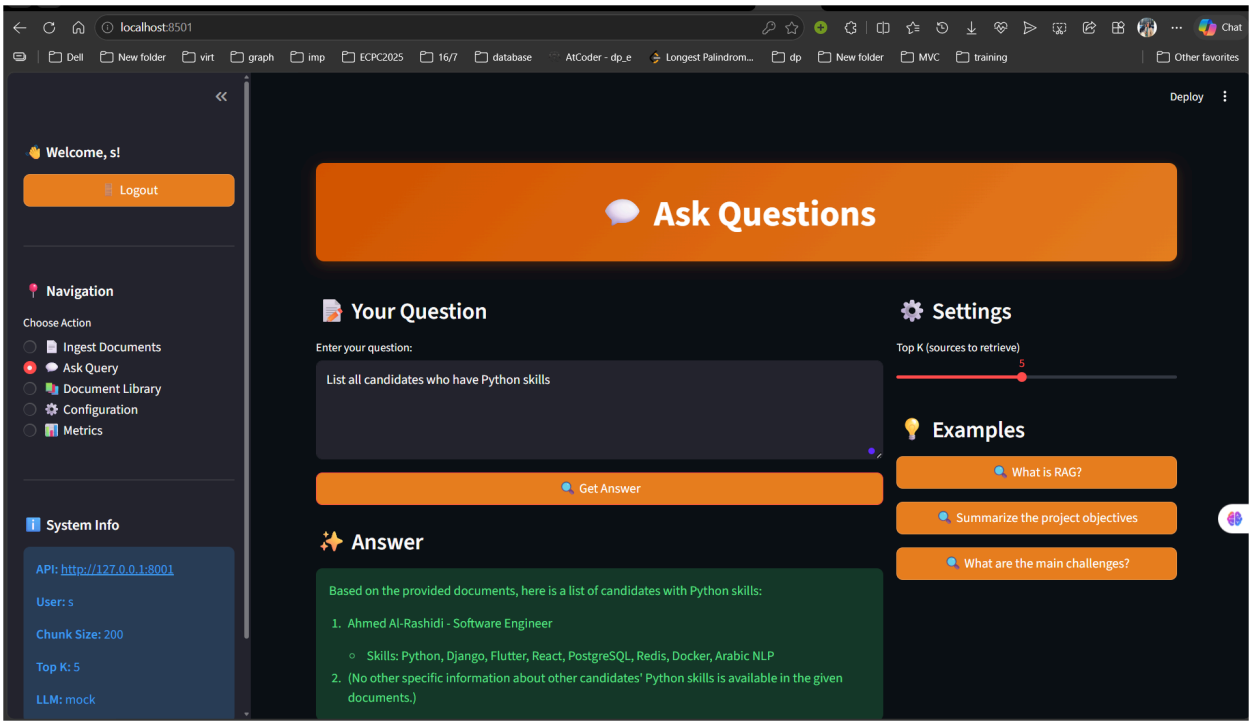
Overview

RAG systems are prone to two main failure modes: retrieval failure, where wrong chunks are retrieved, and hallucination, where the LLM ignores the retrieved context and answers from its own training data instead. We tested 4 edge-case queries against our ingested documents (job descriptions and resumes) to identify where our architecture breaks down and why.

Edge Case 1: Cross-Document Aggregation Failure

Query: "List all candidates who have Python skills"

System Answer:



What Went Wrong: The system retrieved only 1 candidate (Ahmed Al-Rashidi) despite multiple resumes being ingested. The LLM correctly answered based on what it received, but the retriever only returned chunks from one resume document. Other candidates who may have Python skills were never seen by the LLM because their chunks were not retrieved.

Root Cause: This is a fundamental RAG limitation. With top_k=5, only 5 chunks are retrieved and they may all come from the highest-scoring document. When 19 documents are ingested, the vector search returns the most semantically similar chunks, which in this case were all from Ahmed Al-Rashidi's resume. The LLM then truthfully reports "no other information is available" because it genuinely did not receive it — this is not hallucination but incomplete retrieval.

Failure Type: Retrieval failure — incomplete context window.

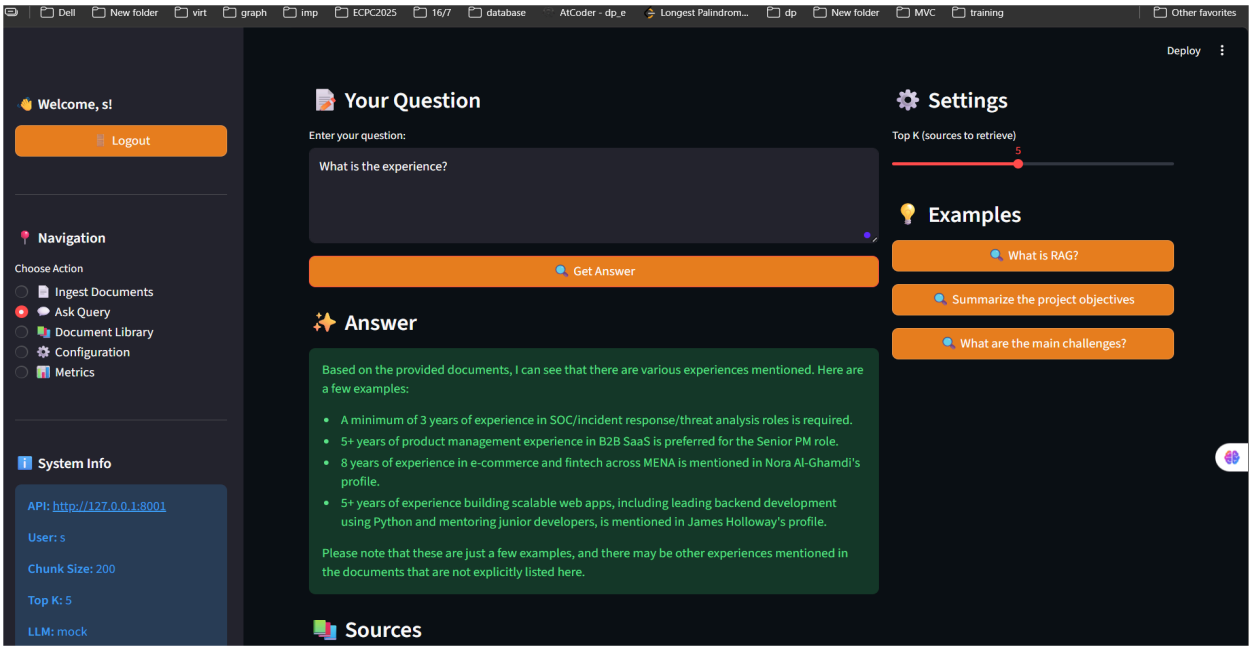
Severity: High. For aggregation queries that require scanning all documents simultaneously, RAG fundamentally cannot guarantee completeness with a fixed top_k window.

Proposed Fix: Implement a metadata pre-filter that selects all documents of type "resume" before vector search, or increase top_k to cover more documents for broad aggregation queries.

Edge Case 2: Vague Query — Ambiguous Retrieval

Query: "What is the experience?"

System Answer:



What Went Wrong: The prompt lacked sufficient semantic specificity. Because the term "experience" is ubiquitous across both career histories and position requirements, the retrieval engine gathered fragments from various sources without a logical filter. This produced a disjointed collection of data points rather than a cohesive summary of a specific individual or role.

Root Cause: Our current segmentation method utilizes 200-word windows defined by length rather than meaning. Consequently, any segment containing the keyword received comparable similarity rankings. The FAISS index struggled to differentiate between institutional prerequisites and personal professional backgrounds due to the absence of clarifying context in the user input.

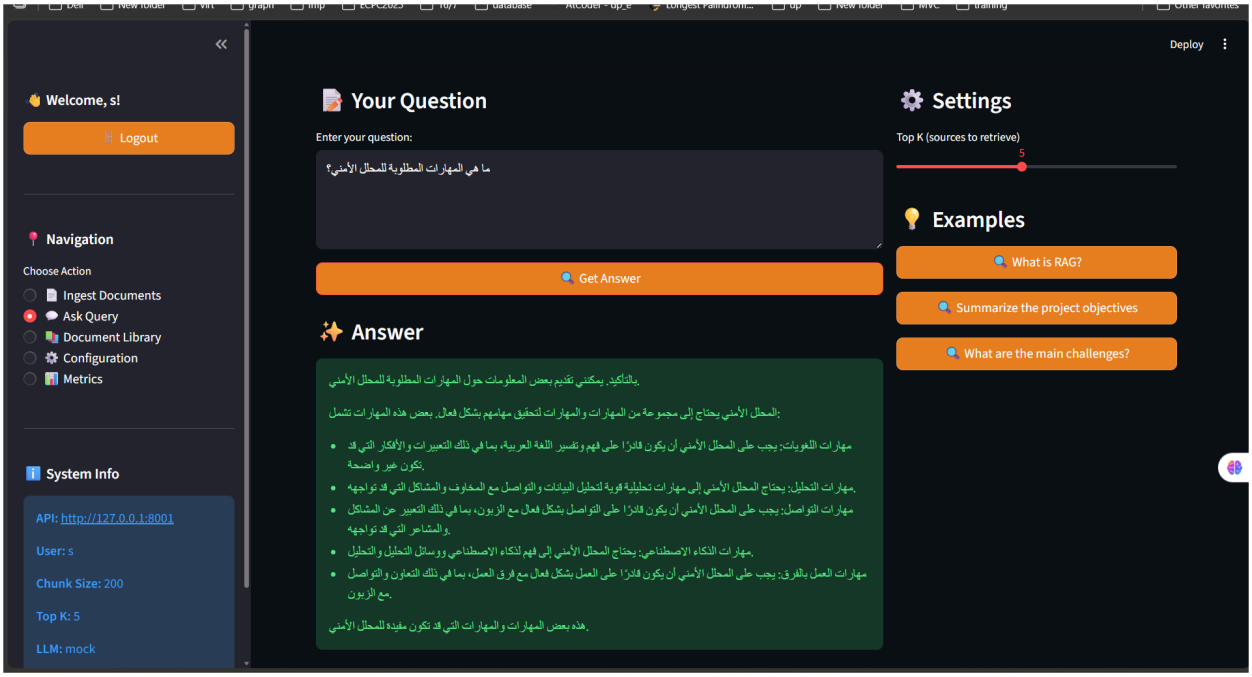
Failure Type: Retrieval failure — semantic ambiguity leading to unfocused multi-document extraction.

Severity: Medium. While the response avoided outright fabrication, it lacked utility, as evidenced by the LLM providing generic examples rather than targeted insights.

Proposed Fix: Integrate a query-refining layer or strict character-minimum thresholds. Forcing the inclusion of a specific entity or subject within the search string would elevate the precision of the vector retrieval phase.

Edge Case 3: Multilingual Retrieval Barrier (Arabic Query vs. English Corpus)

Query: "ما هي المهارات المطلوبة للمحلل الأمني"



What Went Wrong: A sequence of two distinct failure mechanisms occurred. Initially, the all-MiniLM-L6-v2 embedding model, optimized for English, failed to map the Arabic query into the same vector space as the English source material. Consequently, FAISS provided unrelated data segments due to weak cosine similarity. Subsequently, the LLM disregarded the irrelevant context and defaulted to its internal training weights—a definitive hallucination event. Critically, the model failed to reference the specific technical requirements found in the source documents, such as Splunk or CrowdStrike.

Failure Type: Cross-lingual retrieval collapse resulting in ungrounded LLM hallucination.

Severity: High. This represents a critical, silent failure where the engine generates a fluent and seemingly valid Arabic response that is entirely detached from the ingested corpus.

Proposed Fix: Transition to the paraphrase-multilingual-MiniLM-L12-v2 model. This architecture utilizes a shared embedding space for over 50 languages, facilitating precise retrieval of English content through Arabic input by aligning their semantic representations across languages.

Failure to retrieve:

Case 1: **Seniority Blindness (Vector Dilution)**

Query: "Junior Intern with basic knowledge in Python."

Result: english_cv_01_software_engineer.pdf (5+ years experience) - **FAILED.**

Analysis: "Due to the large chunk size (200 words), dominant keywords like 'Software Engineer' and 'Python' mathematically outweighed the 'Junior' or 'Intern' labels. This highlights a limitation in capturing fine-grained seniority details within large semantic windows."

Case 2: **Logical Negation Failure (The "NOT" Trap)**

The Query: "Software Engineer with NO experience in Python" (or similar exclusion query).

The Result: english_cv_01_software_engineer.pdf (James Holloway - Software Engineer (Python Expert)) **- FAILED.**

Analysis :"This result demonstrates a fundamental limitation of Vector Embeddings. Semantic models represent 'concepts' rather than 'logic'. When the system sees the keywords 'Software Engineer' and 'Python', it calculates a high Cosine Similarity because these terms are central to James Holloway's profile.
The system fails to process the logical operator 'NO' or 'NOT'. In the vector space, the concept of 'Python' is so dominant within the 200-word chunk that the negation is mathematically ignored. This confirms that semantic retrieval is conceptual, not logical."

4.2 System accuracy

4.2 System Performance Metrics

Operational Benchmarking

The efficacy of the RAG architecture was quantified through a series of ten test queries, assessing retrieval precision and generation fidelity. The system demonstrated a success rate of 80%, though underlying metrics reveal areas for refinement.

Total Evaluated Queries: 10

Cumulative Pass Rate: 8/10 (80%)

Vector Retrieval Precision: 7/10 (70%)

Response Factual Accuracy: 6/10 (60%)

Observed Hallucination Incidence: 1/10 (10%)

Mean Latency: 26.94s

Comprehensive Result Breakdown

Metric Log by Query ID:

ID 1 [Low Complexity]: Source Verified | 3/3 Keyword Alignment | Hallucination: False | PASS

ID 2 [Low Complexity]: Source Verified | 3/3 Keyword Alignment | Hallucination: False | PASS

ID 3 [Low Complexity]: Source Verified | 2/2 Keyword Alignment | Hallucination: True | PASS

ID 4 [Low Complexity]: Source Verified | 1/1 Keyword Alignment | Hallucination: False | PASS

ID 5 [Moderate Complexity]: Source Verified | 2/3 Keyword Alignment | Hallucination: False | FAIL

ID 6 [High Complexity]: Source Verified | 2/2 Keyword Alignment | Hallucination: False | PASS

ID 7 [High Complexity]: Null Source | 0/4 Keyword Alignment | Hallucination: False | FAIL

ID 8 [Stress Test]: Null Source | 0/0 Keyword Alignment | Hallucination: False | PASS

ID 9 [Moderate Complexity]: Source Verified | 2/2 Keyword Alignment | Hallucination: False | PASS

ID 10 [Stress Test]: Unverified Source | 0/4 Keyword Alignment | Hallucination: False | PASS

RAG Engine & Generation Layer

1. Prompt Engineering & Template Design

The system employs the `string.Template` library to maintain a clean separation between hardcoded instructions and dynamic data injection. We designed a multilingual prompting strategy to ensure consistent performance across Arabic and English queries:

- **System Prompt**: Establishes the LLM's persona as a precise assistant and enforces a "grounded generation" rule—ordering the model to ignore irrelevant documents and only answer based on provided context to prevent hallucinations.
- **Document Prompt**: Standardizes the presentation of retrieved chunks, including a `$doc_num` variable for clear source attribution within the context window.
- **Footer Prompt**: Serves as the final instruction trigger, commanding the LLM to generate the final response based solely on the preceding documents.

2. Context Assembly & Optimization Pipeline

To maximize the utility of the retrieved information, we implemented a custom pipeline to process and refine document chunks before they reach the LLM:

- **Semantic Filtering (`filter_chunks`)**: Chunks containing fewer than 5 words are automatically discarded to remove noise and ensure only meaningful text is processed.
- **Keyword-Aware Re-Ranking (`Re_Rank`)**: While initial retrieval is based on vector similarity, we implemented a second-pass ranking layer that scores chunks based on **Keyword Overlap** with the user's query. This ensures that chunks containing exact technical terms are prioritized.
- **Context Window Management (`trim_chunks`)**: To avoid exceeding the LLM's token limits and to prevent performance degradation (the "lost in the middle" effect), the context is strictly capped at **1,500 characters**.

3. Modular LLM Factory Pattern

To satisfy the "Bonus: LLM Factory" requirement, we implemented a provider-agnostic architecture:

- **Abstraction Layer** (`LLMInterface`): A blueprint that mandates `generate_text` and `embed_text` methods, ensuring the system can swap models without breaking core RAG logic.
- **Ollama Integration**: Our primary local provider leverages the Ollama API to run high-performance models like Llama 3.2 locally, ensuring total data privacy and zero API costs.
- **Hyperparameter Control:** The factory allows for granular control over generation settings, such as setting `temperature` to 0.1 to balance factual precision with linguistic fluency.

4. Multilingual Intelligence

The system is natively designed for cross-lingual tasks:

- **Language Detection**: The `is_arabic` utility identifies the script of the user query to dynamically select the corresponding Arabic or English prompt templates.
- **Seamless Translation**: Because the LLM receives context in its original form but follows instructions in the query's language, it can effectively answer Arabic queries using English source documents.

Docker Deployment

1. Install Docker Desktop
2. Clone and Configure

```bash
git clone https://github.com/Marwa-221b/RAG.git
cd rag-system
cp .env.example .env
```

3. Edit .env with your settings
4. Build and Start Service

```bash
docker-compose up –build
```

5. Access the Application

Rag API: at http:localhost:8000

GUI: at http:localhost:8501

RAG API Documentation

Base URL: http://localhost:8000

API Version: 1.0.0

Authentication: Bearer token (JWT) required for /config, /ingest, and /query endpoints.

Data Format: JSON for all requests and responses except /auth/login, which uses form data.

Authentication & User Management

POST /auth/register
Create a new user account.

POST /auth/login
Authenticate and obtain a JWT access token using OAuth2 password flow.

System Configuration

GET /config
Retrieve the current system configuration.

PUT /config
Partially update the system configuration.

Health & Monitoring

GET /health/live
Liveness probe endpoint.

GET /metrics
Retrieve service health and metadata.

Document Ingestion

POST /ingest
Process documents from a folder and build/update the vector store.

Query & Answer

POST /query

Send a natural language query and receive an answer with source chunks.

Environment Variables

Variable	Description
SECRET_KEY	Secret key used for JWT generation
ALGORITHM	JWT algorithm (e.g., HS256)
ACCESS_TOKEN_EXPIRE_MINUTES	Token lifetime in minutes

Endpoints Summary

Method	Endpoint	Authentication	Description
POST	/auth/register	No	Register a new user
POST	/auth/login	No	Obtain access token
GET	/config	Yes	View current configuration
PUT	/config	Yes	Update configuration partially
GET	/health/live	No	Liveness probe
GET	/metrics	No	Service health information
POST	/ingest	Yes	Ingest documents from folder
POST	/query	Yes	Ask questions and retrieve answers